Phân tích cơ chế tiền và giải ngân SqrtFund

Ngày đối chiếu: 24/10/2026. Phạm vi: source code local hiện tại; không xác nhận deployment Vercel/Fuji đang chạy cùng phiên bản, không thay đổi contract hoặc gửi giao dịch. Đây là phân tích cơ chế MVP, không phải kết luận audit bảo mật.

1. Những phần đã xây dựng

- Factory nhiều Organizer: Platform Owner cấp/thu hồi quyền tạo Round. Mỗi Round có admin là chính ví tạo, token và sổ tiền độc lập.
- StudentRegistry chung: xác thực sinh viên trên blockchain để đăng ký dự án và đóng góp. Backend OTP/email và ràng buộc email–ví phục vụ cấp chứng thực, không giữ quỹ.
- Luồng tạo Round có nạp quỹ trước: frontend dùng Factory deposit rồi `createFundedRound`.
- Đăng ký và duyệt/từ chối dự án; góp một hoặc nhiều dự án trong một giao dịch.
- Tính matching QF bằng số nguyên, chia theo tỷ lệ khi thiếu quỹ, áp trần từng dự án.
- Chốt Round, chủ dự án tự claim toàn bộ khoản được nhận, admin rút phần quỹ đối ứng còn dư.
- Hủy Round trước khi chốt: người góp tự refund, admin rút quỹ đối ứng.
- Xếp hạng hiển thị theo số ví ủng hộ → điểm QF thô → tiền góp trực tiếp; bằng cả ba thì đồng hạng. Xếp hạng không được contract dùng để chia tiền.
- Thông báo Socket.IO và keeper sau 24 giờ đã có code; hiệu lực vận hành phụ thuộc service/secret/configuration. Không suy ra chúng đã bật ở production chỉ vì file tồn tại.

2. Tiền gì, nằm ở đâu?

Token demo là MockUSDC 6 chữ số thập phân trên Fuji: 1 USDC tương ứng 1.000.000 đơn vị token. Đây là token thử nghiệm của dự án, không mặc nhiên là USDC thật được bảo chứng. Gas giao dịch trả riêng bằng AVAX.

Hai khoản tiền được ghi sổ riêng trong Round:

- `pool`: quỹ đối ứng do bên tổ chức nạp.
- `totalDonated` của mỗi dự án: tiền sinh viên đóng góp trực tiếp.

Cả hai cùng nằm trong địa chỉ contract Round, không đi qua Supabase, API, Socket.IO hay ví Platform Owner. Cần phân biệt sổ kế toán và số dư ERC20 thực tế: chuyển token trực tiếp vào địa chỉ contract mà không gọi hàm deposit/donate không cập nhật `pool` hoặc khoản góp.

Trước khi tạo Round

1. Organizer approve token cho Factory.
2. Gọi `depositInitialPool(token, amount)` : token chuyển vào Factory, ghi credit theo ví và token.
3. Gọi `createFundedRound(...)` : trừ credit, tạo Round và chuyển đúng quỹ ban đầu vào Round trong cùng giao dịch. Toàn bộ revert nếu có bước thất bại.
4. Credit chưa dùng có thể được chính ví nạp rút bằng `withdrawInitialPool` , kể cả khi sau đó bị thu hồi quyền Organizer.

Frontend yêu cầu có quỹ trước khi tạo. Tuy nhiên contract vẫn công khai `createRound(...)` với quỹ ban đầu bằng 0. Vì vậy “mọi Round bắt buộc có quỹ trước khi tạo” chưa phải ràng buộc tuyệt đối ở contract.

Sau khi tạo Round

Chỉ admin của Round được gọi `depositPool` . Hiện hàm này cho nạp thêm cả sau `endTime` , miễn chưa finalized và chưa cancelled. Do đó matching có thể tiếp tục thay đổi sau khi ngừng nhận đóng góp và trước lúc chốt.

3. Đóng góp và số người ủng hộ

Ví phải được StudentRegistry xác nhận; Round phải trong `[startTime, endTime)` , chưa chốt và chưa hủy; dự án phải được duyệt. Chủ dự án không được tự góp cho chính dự án đó.

- Mỗi lần góp ít nhất `minDonation` .
- Tổng một ví góp vào một dự án không vượt `maxDonation` .
- Tổng ví chi trong Round không vượt `maxPerRound` .
- Một ví góp nhiều lần cho cùng dự án vẫn tính một người ủng hộ.
- Một ví có thể góp cho nhiều dự án trong giới hạn ngân sách.
- `donateMany` cộng dồn ID lặp, kiểm tra tất cả rồi chuyển token một lần; một mục sai làm revert cả giao dịch.

“Người ủng hộ” ở contract thực chất là địa chỉ ví khác nhau, không phải chứng minh tuyệt đối mỗi ví là một con người. Ràng buộc email giảm trùng lặp theo danh tính email nhưng không chứng minh

một người chỉ sở hữu một email.

4. Công thức tính tiền

Gọi `d_ij` là tổng tiền ví j góp cho dự án i, `D_i = Σ d_ij` , `P` là quỹ đối ứng và `b` là `maxMatchBps` .

QF thô

```
raw_i = max((Σ sqrt(d_ij))² − D_i, 0)
R = Σ raw_i của các dự án đã được duyệt
```

Contract cộng tổng tiền theo ví trước khi lấy căn; tách một khoản góp thành nhiều giao dịch không tạo thêm người hoặc tự nhân điểm QF. Một dự án chỉ có một ví góp sẽ có matching thô bằng 0, nhưng vẫn được nhận tiền góp trực tiếp khi chốt và claim hợp lệ.

Triển khai chính xác ở đơn vị micro-USDC:

```
SCALE = 10^12
sumSqrt_i = Σ floor(sqrt(d_ij_raw × SCALE))
raw_i = max(floor(sumSqrt_i² / SCALE) − D_i_raw, 0)
```

Frontend `src/lib/qf.ts` dùng BigInt theo cùng công thức. Vì có căn và chia số nguyên, số thực minh họa có thể lệch nhỏ so với kết quả on-chain.

Phân bổ và trần

```
Nếu R = 0:        base_i = 0
Nếu 0 < R ≤ P:    base_i = raw_i
Nếu R > P:        base_i = floor(P × raw_i / R)

cap = floor(P × b / 10.000)
matching_i = min(base_i, cap)
leftover = P − Σ matching_i
```

`maxMatchBps` là cấu hình của Round: 3000 nghĩa là 30%, 5000 nghĩa là 50%. Không khẳng định mọi Round đều dùng 30%.

Những hệ quả cần hiểu đúng:

- Quỹ lớn hơn nhu cầu thô không khiến hệ thống tự chia hết quỹ: mỗi dự án tối đa nhu cầu thô và trần của nó.
- Quỹ thiếu sẽ chia theo tỷ lệ điểm QF, sau đó vẫn áp trần.
- Phần bị cắt do trần **không được tái phân phối** cho dự án khác.
- Phần dư gồm tiền chưa dùng do nhu cầu thấp, trần và làm tròn.
- Tỷ lệ đáp ứng trên UI là `min(100%, P / R × 100%)`, cắt xuống 2 chữ số thập phân; không phải P chia tổng donation, cũng không phải tỷ lệ quỹ đã giải ngân. Khi R bằng 0, UI quy ước đủ 100% nhu cầu.

5. Ví dụ tính đầy đủ

Giả sử pool = 10 USDC, trần mỗi dự án = 50% pool = 5 USDC.

Dự án	Đóng góp	Tiền trực tiếp	QF thô	Phân bổ trước trần	Matching sau trần	Tổng được claim
A	2 ví, mỗi ví 4 USDC	8	8	5.714285	5	13
B	3 ví, mỗi ví 1 USDC	3	6	4.285714	4.285714	7.285714

Giải thích: QF A = `(sqrt(4)+sqrt(4))²−8 = 8`; QF B = `(1+1+1)²−3 = 6`. Tổng nhu cầu 14 lớn hơn pool 10, nên chia 10 theo tỷ lệ 8:6. A bị trần ở 5. Phần dư là `10−5−4.285714 = 0.714286 USDC`.

```
Tiền vào Round:            10 + 8 + 3 = 21 USDC
Hai chủ dự án được nhận:   13 + 7.285714 = 20.285714 USDC
Admin được rút phần dư:    0.714286 USDC
Tổng các quyền nhận:       21 USDC
```

B xếp trên A theo số ví ủng hộ, nhưng A nhận nhiều tiền hơn. Hạng không phải phần trăm chia thưởng. Ví dụ này giả định hai dự án được duyệt, không hủy và không có token chuyển ngoài các hàm kế toán.

Đối với ảnh trước đó: pool 200, donation 240, matching 193.988699, dư 6.011301. Nếu snapshot này trở thành kết quả chốt, tổng quyền nhận của các dự án là `240+193.988699=433.988699 USDC`, admin rút dư `6.011301`, tổng bằng 440 USDC đã nạp/góp. Không thể tính ngược chính xác từng dự án chỉ từ các tổng trên ảnh; cần dữ liệu đóng góp và trần của Round.

6. Chốt khác giải ngân

Thao tác	Ai gọi	Điều kiện	Token có chuyển ví không?
`finalize()`	Admin trong 24h sau endTime; sau đó bất kỳ ví	Đã hết hạn, chưa chốt, chưa hủy	Không, chỉ cố định kết quả
`claim(projectId)`	Đúng ví chủ dự án	Đã chốt, dự án được duyệt, chưa claim	Có: donation dự án + `payoutOf`
`withdrawLeftover()`	Admin Round	Đã chốt, chưa rút dư	Có: chỉ phần dư quỹ đối ứng

`payoutOf(projectId)` chỉ lưu phần matching; tổng tiền claim là `project.totalDonated + payoutOf(projectId)`. Mỗi dự án claim một lần, trả toàn bộ vào ví chủ dự án. Một ví có hai dự án phải gọi claim cho từng dự án. Không có chia theo giai đoạn, milestone, duyệt hóa đơn hoặc xác minh sử dụng vốn sau nhận tiền.

Admin không cần đợi mọi dự án claim mới được rút phần dư; quyền nhận của các dự án đã được tách khỏi phần dư theo công thức. Không có hàm thông thường cho admin hoặc Platform Owner rút thay toàn bộ tiền dự án.

Nếu chủ dự án không claim hoặc mất quyền kiểm soát ví, code hiện tại không có hạn claim và không có đường admin thu hồi khoản chưa claim. Token tương ứng tiếp tục nằm trong contract. Sau finalized, không thể cancel hoặc hoàn donation qua luồng refund.

Không thấy cơ chế thu phí nền tảng từ khoản claim trong contract. Người gọi vẫn trả gas AVAX; approve, deposit, donate, finalize, claim, refund là các giao dịch riêng. Giao dịch revert sau khi được đưa vào block vẫn có thể tốn gas.

7. Hủy và hoàn tiền

Admin có thể `cancelRound()` bất kỳ lúc nào trước finalized, kể cả sau endTime. Hủy không tự chuyển token ngay.

- Từng người góp gọi `refund(projectId)` hoặc `refundMany(...)`, nhận lại khoản chính ví đó đã góp. Bản ghi được đưa về 0, giảm ngân sách đã dùng và donorCount, nên không thể refund

hai lần.

- Admin gọi `withdrawPoolIfCancelled()`, nhận lại pool một lần. Không cần đợi mọi người refund; sổ tiền góp và pool tách biệt.
- Round hủy không thể tiếp tục donate, finalize hoặc claim matching.
- Không có đường tự hoàn tiền chỉ vì đã hết 24h; người góp chỉ được refund khi Round đã bị hủy.

8. Keeper và thông báo

Keeper có code chờ `endTime + 86400`, đọc finalized/cancelled, simulation, kiểm tra số dư/gas/nonce rồi gửi finalize và chờ receipt. Mặc định dry-run; cần cấu hình ví riêng đủ AVAX và bật service `--send` hoặc workflow GitHub. Workflow có lịch 15 phút nhưng chỉ chạy định kỳ khi `KEEPER_ENABLED=true`; Node service mặc định quét sau mỗi 60 giây kể từ lượt trước hoàn tất.

Chưa kiểm tra trạng thái vận hành production trong lần phân tích này. Đồng hồ đạt 24 giờ không tự sinh giao dịch nếu keeper chưa chạy. Keeper chỉ chốt, không claim hoặc rút tiền thay ví khác. Socket.IO chỉ phát tín hiệu cập nhật giao diện; không quyết định số tiền hay ký giải ngân.

9. Quyền và các giới hạn còn lại

- Platform Owner quản lý quyền Organizer tại Factory. Thu hồi Organizer chỉ chặn tạo Round mới; admin các Round cũ là immutable nên ví đó vẫn duyệt/hủy/chốt và rút dư/pool cũ theo điều kiện.
- Backend/email attester là một phần tin cậy của việc xác minh sinh viên; chúng không phải nguồn số dư hoặc nguồn quyết định matching.
- Contract giới hạn tối đa 50 dự án/ Round và 2 dự án/owner, kể cả dự án bị từ chối vẫn chiếm quota.
- Luồng tiền giả định token ERC20 thông thường như MockUSDC. Factory kiểm tra số token thực nhận khi prefund; QFRound `donate` / `depositPool` không đo chênh số dư để hỗ trợ token thu phí chuyển tiền. Không thể coi contract tương thích tùy ý mọi ERC20.
- Frontend `/my` và `useClaim()` hiện lấy Round mặc định. Dù thông báo đã đọc nhiều Round, màn nhận tiền chưa phải giao diện claim tổng hợp mọi Round; cần bổ sung chọn Round/lịch sử nếu muốn nhận tiền Round cũ ngay trên UI. Contract mỗi Round vẫn có `claim()` riêng.
- Trạng thái frontend và preview phụ thuộc RPC/cache; kết quả sau chốt lấy `payoutOf`, không được dùng preview thay kết quả chính thức nếu đọc payout thất bại.

10. Tệp nguồn đối chiếu

- `contracts/src/RoundFactory.sol` : quyền và tiền nạp trước tạo Round.
- `contracts/src/PrefundedQFRound.sol` : gán pool ban đầu.
- `contracts/src/QFRound.sol` : donation, QF, cap, finalize, claim, leftover, cancel/refund.
- `contracts/src/MockUSDC.sol` , `StudentRegistry.sol` : token demo và xác minh quyền sinh viên.
- `src/lib/qf.ts` , `src/lib/roundStats.ts` : tính preview và thống kê.
- `src/hooks/useProjects.ts` : đọc kết quả settlement `payoutOf` sau finalized.
- `src/components/admin/CreateRoundForm.tsx` , `RoundControls.tsx` : luồng nạp/chốt/rút trên UI.
- `src/components/leaderboard/model.ts` : xếp hạng frontend, độc lập settlement.
- `src/hooks/useClaim.ts` , `src/views/my.tsx` : chủ dự án ký claim, hiện theo Round mặc định.
- `scripts/keeper/finalize.mjs` , `.github/workflows/finalize-keeper.yml` : cơ chế chạy keeper tùy chọn.

11. Kiểm chứng trong lần phân tích

Đã chạy 66 test thuộc 7 file: `qf.test.ts` , `round-funding-stats.test.ts` , `round-funding-flow.test.ts` , `round-refunds.test.ts` , `finalized-leaderboard.test.ts` , `finalize-keeper.test.ts` , `leaderboard-presentation.test.ts` ; tất cả đạt. Ví dụ A/B được tính lại trực tiếp bằng `computeMatchingBigInt` trong code, trả matching 5.000000 và 4.285714, dư 0.714286 USDC.

Đây là test local/mocked và đối chiếu source. Không có giao dịch giải ngân/hoàn tiền thật, không kiểm tra vận hành production hoặc audit contract triển khai trên chain trong lần này.